Assignment Beta – Question 4
Evaluating an LLM sentiment classifier

Syed Ukkashah · 23K-0055 · Notebook: Assignment_Beta_Question_no_4_23K-0055.ipynb

300 reviews, three classes. Confusion matrices have rows = actual and columns = predicted. All metrics were computed by hand in the notebook; scikit-learn and NLTK were used only to cross-check them, and they agree to 4 decimal places.

Stage 1: zero-shot baseline

Stage 1	Pred Positive	Pred Negative	Pred Neutral	Row total
Actual Positive	100	5	15	120
Actual Negative	8	80	12	100
Actual Neutral	20	10	50	80
Column total	128	95	77	300

1. Overall accuracy with a 95% confidence interval

```
accuracy = trace / n = 230/300 = 0.7667
SE = √(p(1 − p)/n) = √(0.7667 · 0.2333 / 300) = 0.0244
95% Wald CI   = 0.7667 ± 1.96 · 0.0244 = [0.7188, 0.8145]
95% Wilson CI = [0.7156, 0.8110]
```

Accuracy is 0.7667 (76.7%), 95% CI [0.719, 0.815] (Wald, normal approximation). The Wilson interval, which behaves better near 0 or 1, is almost the same at n = 300.

2. Precision, recall and F1 per class

Class	Precision = TP / col	Recall = TP / row	F1 = 2PR / (P + R)
Positive	100/128 = 0.7812	100/120 = 0.8333	0.8065
Negative	80/95 = 0.8421	80/100 = 0.8000	0.8205
Neutral	50/77 = 0.6494	50/80 = 0.6250	0.6369

3. Macro, weighted and micro F1

```
macro F1    = (0.8065 + 0.8205 + 0.6369) / 3 = 0.7546
weighted F1 = (120·0.8065 + 100·0.8205 + 80·0.6369) / 300 = 0.7659
micro F1    = 2·TP / (2·TP + FP + FN) = 2·230 / (2·230 + 70 + 70) = 0.7667
```

Macro F1 weights every class equally; weighted F1 weights by support (120, 100, 80). In single-label multi-class classification every error is one false positive (for the predicted class) and one false negative (for the true class), so micro precision = micro recall = micro F1 = accuracy.

4. Interpreting Stage 1

Where the model fails:

- Neutral is the weak class (F1 0.637 vs about 0.81 for the others). Recall is 50/80 = 0.625, so more than a third of neutral reviews get a polar label, and 20 of those 30 misses are called Positive. Precision is also the lowest (50/77 = 0.649), because 15 Positive and 12 Negative reviews are pushed into Neutral.
- Positive ↔ Neutral is the main confusion: 20 + 15 = 35 of the 70 errors (50%). Negative ↔ Positive mistakes are rare (5 + 8 = 13), so the model reads polarity well. What it gets wrong is where "no clear opinion" begins.
- The prompt gives only the label names, so the model has to guess what Neutral means. Mixed reviews ("late delivery but good product") are the most ambiguous, and LLMs tend to lean positive on politely worded text.

Recommended changes:

1. Define every label in the prompt, especially Neutral = factual, mixed, or no clear opinion.
2. Add few-shot examples (two per class), deliberately including borderline mixed reviews.
3. Constrain the output to exactly one label (or a JSON field), so parsing never adds errors.
4. Test sensitivity to example ordering, and if needed use label log-probabilities with a tuned Neutral threshold.
5. Iterate on errors: read the 35 Positive/Neutral confusions and turn the common patterns into examples.

The Stage 2 prompt implements changes 1 and 2.

5. Stage 2: improved prompt, and comparison

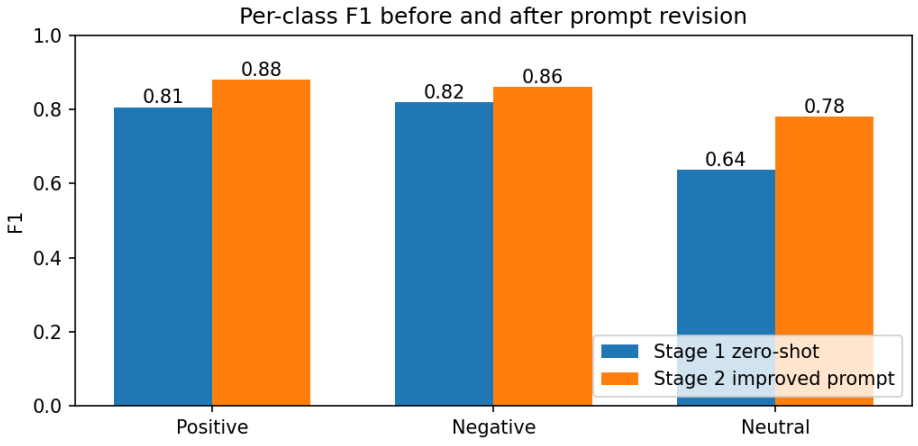
Stage 2	Pred Positive	Pred Negative	Pred Neutral	Row total
Actual Positive	106	4	10	120
Actual Negative	6	84	10	100
Actual Neutral	9	7	64	80
Column total	121	95	84	300

```
accuracy = trace / n = 254/300 = 0.8467
SE = √(p(1 − p)/n) = √(0.8467 · 0.1533 / 300) = 0.0208
95% Wald CI   = 0.8467 ± 1.96 · 0.0208 = [0.8059, 0.8874]
95% Wilson CI = [0.8015, 0.8830]
```

Class	Precision = TP / col	Recall = TP / row	F1 = 2PR / (P + R)
Positive	106/121 = 0.8760	106/120 = 0.8833	0.8797
Negative	84/95 = 0.8842	84/100 = 0.8400	0.8615
Neutral	64/84 = 0.7619	64/80 = 0.8000	0.7805

```
macro F1    = (0.8797 + 0.8615 + 0.7805) / 3 = 0.8406
weighted F1 = (120·0.8797 + 100·0.8615 + 80·0.7805) / 300 = 0.8472
micro F1    = 2·TP / (2·TP + FP + FN) = 2·254 / (2·254 + 46 + 46) = 0.8467
```

Metric	Stage 1 (zero-shot)	Stage 2 (definition + few-shot)	Change
Accuracy	0.7667	0.8467	+0.0800
95% CI (Wald)	[0.7188, 0.8145]	[0.8059, 0.8874]	
F1 Positive	0.8065	0.8797	+0.0732
F1 Negative	0.8205	0.8615	+0.0410
F1 Neutral	0.6369	0.7805	+0.1435
Macro F1	0.7546	0.8406	+0.0859
Weighted F1	0.7659	0.8472	+0.0812
Micro F1	0.7667	0.8467	+0.0800



- Accuracy rises from 0.767 to 0.847 (+8.0 points). The two 95% intervals barely overlap, so the gain is very unlikely to be noise. (A paired McNemar test would be the proper check, but it needs per-review predictions, which are not given.)
- Neutral improves the most (F1 0.637 → 0.780): recall goes from 50/80 to 64/80, and Neutral→Positive errors fall from 20 to 9. This is exactly the error the definition and the borderline examples targeted.
- Positive and Negative improve too, because fewer of them are pushed into Neutral (15 → 10 and 12 → 10). The errors were removed, not just moved around.
- Macro F1 gains more than weighted F1 (+0.086 vs +0.081), because the smallest class, Neutral, gained the most.

6. Prompt stability across five example orderings

```
mean = (0.847 + 0.840 + 0.853 + 0.837 + 0.850) / 5 = 4.227 / 5 = 0.8454
deviations = [+0.0016, -0.0054, +0.0076, -0.0084, +0.0046]
Σ(dev²) = 0.000181
sample SD = √(0.000181 / 4) = 0.0067     (population SD, /5: 0.0060)
```

Result: 0.8454 ± 0.0067

The sample SD (n − 1) is used because the five orderings are a sample of all possible orderings. The spread is about ±0.007 (roughly 2 reviews out of 300), and the whole range (0.837–0.853, 5 reviews) is far smaller than the binomial sampling error of a single run (about ±0.04). Every ordering beats the zero-shot 0.767 by a wide margin. The prompt is stable with respect to example order, so the Stage 2 gain comes from what the prompt says, not from a lucky ordering.

7. Justification quality: BLEU-4, ROUGE and METEOR

Reference: the delivery was late but the product quality is good (10 tokens)

Candidate: the product quality is good but delivery was slow (9 tokens)

BLEU-4

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp(¼\ \Sigma_n \ln p_n), \quad \mathrm{BP} = e\text{^}(1 - r/c) \text{ if } c < r$$

```
p1 = 8/9 = 0.8889
p2 = 5/8 = 0.6250
p3 = 3/7 = 0.4286
p4 = 2/6 = 0.3333
BP = exp(1 − 10/9) = 0.8948
BLEU-4 = 0.8948 · exp(-2.5337 / 4) = 0.4750
```

Matched 4-grams: "the product quality is" and "product quality is good". No smoothing is needed because every p_n is non-zero.

ROUGE-1, ROUGE-2, ROUGE-L

Metric	Overlap	Precision	Recall	F1
ROUGE-1	8 unigrams	8/9 = 0.8889	8/10 = 0.8000	0.8421
ROUGE-2	5 bigrams	5/8 = 0.6250	5/9 = 0.5556	0.5882
ROUGE-L	LCS = 5	5/9 = 0.5556	5/10 = 0.5000	0.5263

The LCS is "the product quality is good" (5 words). It can follow only one of the two clauses, because they appear in opposite order in the two sentences.

METEOR (exact match, α = 0.9, β = 3, γ = 0.5)

```
matches m = 8  (all candidate words except "slow")
P = 8/9 = 0.8889    R = 8/10 = 0.8000
Fmean = P·R / (0.9·P + 0.1·R) = 0.8081
chunks = 3  [the product quality is good] [but] [delivery was]
penalty = 0.5 · (3/8)³ = 0.0264
METEOR = 0.8081 · (1 − 0.0264) = 0.7868
```

"the" occurs twice in the reference, so both alignments were tried and the one with the fewest chunks was kept, as METEOR specifies. NLTK's METEOR, which also tries stem and WordNet-synonym matches, gives the same 0.7868: it does not treat late and slow as synonyms.

BLEU-4	ROUGE-1 F1	ROUGE-2 F1	ROUGE-L F1	METEOR
0.4750	0.8421	0.5882	0.5263	0.7868

Interpretation:

The candidate keeps the meaning of the reference (one complaint, one compliment) but swaps the two clauses and says slow instead of late. ROUGE-1 is high because 8 of the 9 words are shared. BLEU-4, ROUGE-2 and ROUGE-L are much lower because they reward word order, and the clause swap breaks the bigrams around "but". METEOR stays high because it is recall-weighted and only lightly penalises the 3 chunks. So the n-gram metrics under-rate a justification a human would accept. For free-text explanations, a semantic metric (BERTScore, an LLM judge) or a human check is more appropriate.

8. Perplexity of the generated justification

$$\mathrm{PPL} = \exp(-(1/N)\,\Sigma_i \ln p_i)$$

Token	the	product	quality	is	good	but	delivery	was	slow
p	0.60	0.40	0.50	0.80	0.70	0.30	0.25	0.50	0.20
ln p	-0.5108	-0.9163	-0.6931	-0.2231	-0.3567	-1.2040	-1.3863	-0.6931	-1.6094

```
Σ ln p = -7.5929
average negative log-likelihood = 7.5929 / 9 = 0.8437
PPL = exp(0.8437) = 2.3249     (= 1 / geometric mean of p = 1 / 0.4301)
```

A perplexity of about 2.32 means the model was, on average, as uncertain as if it were choosing uniformly among about 2.3 tokens at each step. That is fairly confident for free text. The confidence is uneven, though. The model was surest about is (0.80) and good (0.70), and least sure about the second half: but (0.30), delivery (0.25), slow (0.20). That is exactly where it departs from the reference (the clause order, and slow vs late). Low perplexity measures fluency and confidence, not correctness: a justification can be confident and still wrong, so perplexity should be read alongside the overlap metrics in part 7, not instead of them.